Appendices to "Assortment Optimization with Customer-Centric Fairness"

Appendix A presents the algorithms, theoretical results, and numerical experiments omitted from the main text. Appendix B provides the proofs.

Appendix A. Supplementary Algorithms

Throughout this appendix, write $D_i = |\mathcal{S}_i^D|$ and $R_i^D = R_i(\mathcal{S}_i^D)$. Customer types retain the ordering $D_1 \le \cdots \le D_M$ used in the main text. We retain the convention in the main text of choosing the revenue-ordered fully customized optimum with the smallest cardinality. All inequalities describing monotonicity are understood in the weak sense. An indexed loop or sum is empty when its lower limit exceeds its upper limit.

A.1. Polynomial Algorithm for Solving Problem (DFA-2) in the Two-type Case

We reformulate the subproblem for each group i as follows:

$$\begin{aligned} R_i^*(C) &= \max_R\{R | R \le \frac{\sum_{j\in\mathcal{S}_i} r_j v_{ij}}{1+\sum_{j\in\mathcal{S}_i} v_{ij}},\ \exists \mathcal{S}_i, |\mathcal{S}_i| = C\} \\ &= \max_R\{R | R \le \sum_{j\in\mathcal{S}_i} (r_j - R) v_{ij},\ \exists \mathcal{S}_i, |\mathcal{S}_i| = C\} \\ &= \max_R\{R | R \le \max_{\mathcal{S}_i:|\mathcal{S}_i|=C} \sum_{j\in\mathcal{S}_i} (r_j - R) v_{ij}\}. \end{aligned} \tag{A.1}$$

Define the linear functions $h_{ij}(R) = v_{ij}(r_j - R)$, where $i = 1, 2$, $j \in \mathcal{N}$. In the last equation, for a fixed R, the value of the subproblem $\max_{\mathcal{S}_i:|\mathcal{S}_i|=C} \sum_{j\in\mathcal{S}_i}(r_j - R)v_{ij}$ corresponds to the sum of the C largest values of $h_{ij}(R)$, $j \in \mathcal{N}$. An important observation is that any two functions $h_{ij}(R)$ and $h_{ik}(R)$ with $j, k \in \mathcal{N}$ and $v_{ij} \ne v_{ik}$ intersect at most once since they are linear in R, indicating that their relative order only changes at their intersection point.

Let Z_i represent the number of distinct positive intersection coordinates between nonparallel functions $h_{ij}(R)$ and $h_{ik}(R)$, $j, k \in \mathcal{N}$. Collect these coordinates and relabel them in increasing order: $0 < I_i^{(1)} < I_i^{(2)} < \cdots < I_i^{(Z_i)}$. Define $I_i^{(0)} = 0$ and $I_i^{(Z_i+1)} = +\infty$. If there is no positive intersection, then $Z_i = 0$ and the only interval is $[0, +\infty)$. Based on our definition, for any $R \in [I_i^{(z)}, I_i^{(z+1)})$, we observe that the ordering of $h_{ij}(R)$, $j \in \mathcal{N}$ remains the same. Therefore, there is a well-defined permutation $\sigma_i^{(z)} = (j_1^{i,z}, \ldots, j_l^{i,z}, \ldots, j_N^{i,z})$ of $j \in \mathcal{N}$ such that $h_{ij_l^{i,z}}(R)$, $l \in \mathcal{N}$ is in a decreasing order for any $R \in [I_i^{(z)}, I_i^{(z+1)})$. That is, $h_{ij_1^{i,z}}(R) \ge h_{ij_2^{i,z}}(R) \ge \cdots \ge h_{ij_l^{i,z}}(R) \ge \cdots \ge h_{ij_N^{i,z}}(R)$. Thus, $\sigma_i^{(z)}$ is defined for every $z = 0, \ldots, Z_i$. Let the assortment set $\mathcal{B}_i^{(z)}(C)$ to be $\{j_1^{i,z}, \ldots, j_l^{i,z}, \ldots, j_C^{i,z}\}$ containing the first C elements of $\sigma_i^{(z)}$. Consequently, we have the following equation:

$$\max_{\mathcal{S}_i:|\mathcal{S}_i|=C} \sum_{j\in\mathcal{S}_i} h_{ij}(R) = \sum_{j\in\mathcal{B}_i^{(z)}(C)} h_{ij}(R) \tag{A.2}$$

for any $R \in [I_i^{(z)}, I_i^{(z+1)})$. Then the optimal assortment of Problem (A.1) is in $\{\mathcal{B}_i^{(z)}(C)\}_{z=0}^{Z_i}$, and we can solve Problem (A.1) by searching for the maximal value of R. For the efficient construction of $I_i^{(z)}$ and $\sigma_i^{(z)}$, see Rusmevichientong et al. (2010) for details.

Given the analysis above, it is feasible to solve Problem (DFA-2) by enumerating the value of $|\mathcal{S}_1|$ and then solving the corresponding inner problem (A.1) independently. However, the calculation can be simplified further by leveraging the unimodal property established in Lemma 2. We next illustrate this in Algorithm A.1. Instead of following a sequential approach that first solves the inner problem and then enumerates $|\mathcal{S}_1|$, our algorithm is designed to consider both layers jointly, integrating the structure of the inner problem with the overall optimization process to improve efficiency.

The basic logic of Algorithm A.1 is to enumerate C from its lower bound $|\mathcal{S}_1^D|$ to upper bound $|\mathcal{S}_2^D| - \alpha_{2,1}$ while simultaneously tracking the interval to which $R_i^*(C)$ belongs based on the result of $R_i^*(C-1)$. We use the position of $R_i^*(C)$ to represent the value of z such that $R_i^*(C) \in [I_i^{(z)}, I_i^{(z+1)})$. The key idea of the algorithm is detailed below.

Algorithm A.1 Polynomial algorithm of Problem (DFA-2)

Require: r_j, $\mathcal{S}_i^D$, λ_i, $\alpha_{1,2}$, $\alpha_{2,1}$, v_{ij} for $i = 1, 2$, $j \in \mathcal{N}$.

Step 1: If $|\mathcal{S}_2^D| - |\mathcal{S}_1^D| \leq \alpha_{2,1}$, return $(\mathcal{S}_1^D, \mathcal{S}_2^D)$ and stop. Otherwise, for each $i = 1, 2$, compute the intervals $[I_i^{(z)}, I_i^{(z+1)})$ and successively reorder the products to obtain $\sigma_i^{(z)}$, $z = 0, \ldots, Z_i$. Then initiate $C = |\mathcal{S}_1^D|$. Let $\bar{z}_1 = \max\{z \in \{0, \ldots, Z_1\} : I_1^{(z)} \leq R_1(\mathcal{S}_1^D)\}$ and $\underline{z}_2 = 0$. Let $\Pi_{max} = -\infty$.

Step 2: Repeat the following process while $C \leq |\mathcal{S}_2^D| - \alpha_{2,1}$:

Step 2.1: Set $z_1 = \bar{z}_1$. While $z_1 > 0$ and $\sum_{j \in \mathcal{B}_1^{(z_1)}(C)} h_{1j}(I_1^{(z_1)}) < I_1^{(z_1)}$, update $z_1 = z_1 - 1$. Then update $\bar{z}_1 = z_1$. Similarly, set $z_2 = \underline{z}_2$. While $z_2 < Z_2$ and $\sum_{j \in \mathcal{B}_2^{(z_2+1)}(C+\alpha_{2,1})} h_{2j}(I_2^{(z_2+1)}) \geq I_2^{(z_2+1)}$, update $z_2 = z_2 + 1$. Then update $\underline{z}_2 = z_2$.

Step 2.2: Set $\Pi_{temp} = \lambda_1 \sum_{j \in \mathcal{B}_1^{(\bar{z}_1)}(C)} r_j \frac{v_{1j}}{1+\sum_{l \in \mathcal{B}_1^{(\bar{z}_1)}(C)} v_{1l}} + \lambda_2 \sum_{j \in \mathcal{B}_2^{(\underline{z}_2)}(C+\alpha_{2,1})} r_j \frac{v_{2j}}{1+\sum_{l \in \mathcal{B}_2^{(\underline{z}_2)}(C+\alpha_{2,1})} v_{2l}}$. If $\Pi_{temp} > \Pi_{max}$, update $\mathcal{S}_1^F = \mathcal{B}_1^{(\bar{z}_1)}(C)$, $\mathcal{S}_2^F = \mathcal{B}_2^{(\underline{z}_2)}(C + \alpha_{2,1})$ and update Π_{max} as Π_{temp}.

Step 2.3: Update $C = C + 1$.

Ensure: $\mathcal{S}_1^F$, $\mathcal{S}_2^F$.

First, we show that the search for the position of $R_i^*(C)$ can be conducted by gradually increasing or decreasing z. Notice that $\max_{\mathcal{S}_i : |\mathcal{S}_i| = C} \sum_{j \in \mathcal{S}_i} h_{ij}(R)$ decreases in R. Based on the reformulated Problem (A.1), the correct position is the largest $z \in \{0, \ldots, Z_i\}$ satisfying $\sum_{j \in \mathcal{B}_i^{(z)}(C)} h_{ij}(I_i^{(z)}) \geq I_i^{(z)}$. Therefore, we can decrease or increase z to search for this position. The detailed proof is provided in the appendix.

Second, leveraging the unimodal property established in Lemma 2, the position of $R_i^*(C)$ can be located efficiently using the known position of $R_i^*(C-1)$. Assuming that we have already found the position of $R_i^*(C-1)$ denoted as z', which means $R_i^*(C-1) \in [I_i^{(z')}, I_i^{(z'+1)})$, and we take the case $C \leq |\mathcal{S}_i^D|$ as an example. Because $R_i^*(C)$ has the property of increasing in terms of C when $C \leq |\mathcal{S}_i^D|$, it leads to $R_i^*(C) \geq R_i^*(C-1) \geq I_i^{(z')}$. So we only need to increase z starting from z' while $z < Z_i$ and we stop when that test fails or $z = Z_i$. A similar process can be conducted for the case when $C > |\mathcal{S}_i^D|$. This approach avoids repeatedly enumerating z from 0 to Z_i for every C, significantly improving the computational efficiency. This process corresponds to Step 2.1 of Algorithm A.1, where $\bar{z}_1$ and $\underline{z}_2$ record the position of $R_1^*(C)$ and $R_2^*(C + \alpha_{2,1})$, respectively.

Consequently, we can determine the optimal assortment under fixed cardinality directly after identifying the position of $R_i^*(C)$ based on Equation (A.2). By gradually increasing C and keeping track of the current maximum revenue Π_{max}, the assortment that yields the highest revenue corresponds to the exact optimal solution of Problem (DFA-2). Note that given the interval information, the scan has time complexity $O(N^2)$, which matches the complexity of solving the subproblem (A.1) for a single group with a fixed cardinality (Rusmevichientong et al. 2010). This indicates that our algorithm incurs no additional computational complexity even after involving more than one customer group and capturing fairness interactions between groups.

A.2. Optional Shortest-Path Preprocessing before Algorithm 1

Because fairness depends only on assortment sizes, let $\boldsymbol{C} = (C_i)_{i\in\mathcal{M}}$ collect the cardinalities offered to all customer groups. The original fairness constraints permit exactly the cardinality vectors in

$$\mathcal{F}_0 = \left\{\boldsymbol{C} \in \{0,1,\dots,N\}^M : C_{i_1} - C_{i_2} \le \alpha_{i_1,i_2}, \quad \forall i_1, i_2 \in \mathcal{M},\ i_1 \ne i_2\right\}.$$

We seek to describe this same set using as few pairwise constraints as possible, which is equivalent to solving

$$\begin{aligned}(\mathcal{A}', \boldsymbol{d}_{\mathcal{A}'}) \in \arg\min_{\mathcal{A}, \boldsymbol{x}_{\mathcal{A}}} \quad & |\mathcal{A}| \\ \text{s.t.} \quad & \left\{\boldsymbol{C} \in \{0,1,\dots,N\}^M : C_{i_1} - C_{i_2} \le x_{i_1,i_2},\ \forall (i_1,i_2) \in \mathcal{A}\right\} = \mathcal{F}_0, \\ & \mathcal{A} \subseteq \{(i_1,i_2) \in \mathcal{M}^2 : i_1 \ne i_2\}, \quad \boldsymbol{x}_{\mathcal{A}} = (x_{i_1,i_2})_{(i_1,i_2)\in\mathcal{A}} \in \mathbb{Z}_+^{|\mathcal{A}|}.\end{aligned}$$

Here, $\mathcal{A}'$ indexes a minimum-cardinality collection of fairness constraints whose feasible cardinality region remains exactly $\mathcal{F}_0$, and $\boldsymbol{d}_{\mathcal{A}'} = (d_{i_1,i_2})_{(i_1,i_2)\in\mathcal{A}'}$ denotes the corresponding tolerance levels. Moreover, we say that a retained tolerance level d_{i_1,i_2} is tight if replacing it by any smaller nonnegative integer alters the feasible cardinality region.

We can obtain an optimal $(\mathcal{A}', \boldsymbol{d}_{\mathcal{A}'})$ through a shortest-path preprocessing procedure, and then replace the original fairness constraints without changing the optimal assortment strategy. In Algorithm A.2, Step 1 constructs a directed graph with non-negative and finite edge values based on the original tolerance levels. Step 2 computes the shortest path over the graph. We use the Floyd-Warshall algorithm (Floyd 1962) since it is particularly suitable for complete graphs and computes all-pairs shortest paths efficiently. This shortest-path computation determines the implied tolerances. Specifically, if there exists a path $i_1 \to k_1 \to k_2 \to \cdots \to i_2$, then the corresponding fairness constraints imply $|\mathcal{S}_{i_1}| - |\mathcal{S}_{i_2}| \le \alpha_{i_1,k_1} + \alpha_{k_1,k_2} + \cdots + \alpha_{k_i,i_2}$. Therefore, the tightest tolerance level of $|\mathcal{S}_{i_1}| - |\mathcal{S}_{i_2}|$ is the shortest-path distance d_{i_1,i_2}. Finally, Step 3 removes all directed constraints that can be implied by other constraints. Step 3.1 retains the minimum number of constraints needed to connect all customer types within each zero-distance equivalence class. Different classes may still have a zero distance in one direction. Step 3.2 therefore considers all constraints between distinct classes with distances below N, including zero distances, and retains a constraint only when its distance cannot be reproduced by passing through a third class. Constraints with $d_{Q,R} = N$ need not be retained, because $0 \le C_i \le N$ already implies $C_{i_1} - C_{i_2} \le N$.

For better understanding, an illustrative example is provided below, where our preprocessing can substantially simplify the constraints.

EXAMPLE 1. Consider a market with $M = 4$ customer groups ordered by a one-dimensional index, such as loyalty tier, income level, or service priority. Suppose the tolerance levels have a clear hierarchical structure: adjacent groups are allowed to differ by at most one product, and the tolerance between two nonadjacent groups is proportional to their index distance. The tolerance matrix is given in Table A.1.

Table A.1 Tolerance levels under a hierarchical group structure

α_{i_1,i_2}	$i_2=1$	$i_2=2$	$i_2=3$	$i_2=4$
$i_1=1$	0	**1**	2	3
$i_1=2$	**1**	0	**1**	2
$i_1=3$	2	**1**	0	**1**
$i_1=4$	3	2	**1**	0

In this case, the tolerance level between any two nonadjacent groups can be implied by the constraints between adjacent groups. For example, $|\mathcal{S}_1| - |\mathcal{S}_4| = (|\mathcal{S}_1| - |\mathcal{S}_2|) + (|\mathcal{S}_2| - |\mathcal{S}_3|) + (|\mathcal{S}_3| - |\mathcal{S}_4|) \le 3$ and thus the constraint $|\mathcal{S}_1| - |\mathcal{S}_4| \le 3$ is redundant. After applying the preprocessing algorithm, we only need to keep the constraints with bold numbers in Table A.1, where the number of fairness constraints can be reduced from $O(M^2)$ to $O(M)$. □

Algorithm A.2 Preprocessing to eliminate redundant fairness constraints

Require: Original fairness constraints and tolerance levels α_{i_1,i_2}.

Step 1: Construct a directed graph with M nodes. For every original constraint $|\mathcal{S}_{i_1}| - |\mathcal{S}_{i_2}| \le \alpha_{i_1,i_2}$, assign length $\min\{N, \alpha_{i_1,i_2}\}$ to the directed edge from node i_1 to node i_2.

Step 2: Apply the Floyd-Warshall algorithm to compute the shortest-path distances between all pairs of nodes. Let d_{i_1,i_2} be the shortest-path distance from i_1 to i_2, and set $d_{i,i} = 0$ for every $i \in \mathcal{M}$.

Step 3: Initialize $\mathcal{A}' \leftarrow \emptyset$. Partition $\mathcal{M}$ into zero-distance equivalence classes according to $i_1 \sim i_2 \iff d_{i_1,i_2} = d_{i_2,i_1} = 0$. Let $\mathcal{Q}$ denote the resulting collection of equivalence classes. For every $Q, R \in \mathcal{Q}$, define $d_{Q,R} := d_{i_1,i_2}$ for any $i_1 \in Q$ and $i_2 \in R$, and choose an arbitrary representative $\mathrm{rep}(Q) \in Q$.

Step 3.1: For every equivalence class $Q = \{q_1, \ldots, q_s\} \in \mathcal{Q}$ with $s \ge 2$, choose an arbitrary cyclic ordering of its nodes, set $q_{s+1} := q_1$, and add $(q_\ell, q_{\ell+1})$, $\ell = 1, \ldots, s$, to $\mathcal{A}'$, with $d_{q_\ell, q_{\ell+1}} = 0$ for all $\ell = 1, \ldots, s$.

Step 3.2: For every two distinct classes $Q, R \in \mathcal{Q}$ satisfying $0 \le d_{Q,R} < N$, if $d_{Q,R} < d_{Q,T} + d_{T,R}, \quad \forall T \in \mathcal{Q} \setminus \{Q, R\}$, add $\big(\mathrm{rep}(Q), \mathrm{rep}(R)\big)$ to $\mathcal{A}'$, with $d_{\mathrm{rep}(Q),\mathrm{rep}(R)} = d_{Q,R}$.

Ensure: $(\mathcal{A}', \boldsymbol{d}_{\mathcal{A}'})$.

LEMMA A.1. *For arbitrary tolerance levels, Algorithm A.2 returns an optimal solution $(\mathcal{A}', \boldsymbol{d}_{\mathcal{A}'})$ to the reduction problem above in $O(M^3)$ time. Moreover, every retained tolerance level d_{i_1,i_2}, $(i_1, i_2) \in \mathcal{A}'$, is tight. Consequently, the reduced region $\big\{\boldsymbol{C} \in \{0, 1, \ldots, N\}^M : C_{i_1} - C_{i_2} \le d_{i_1,i_2},\ \forall (i_1, i_2) \in \mathcal{A}'\big\}$ is exactly the same feasible cardinality region as that in Problem* (DFA-M).

Lemma A.1 allows us to replace the original fairness constraints with the reduced system $(\mathcal{A}', \boldsymbol{d}_{\mathcal{A}'})$ without changing the feasible assortments or the optimal revenue. This optional preprocessing can be applied before Algorithm 1. This replacement preserves the original assortment problem while reducing the number of fairness edges from $O(M^2N)$ to $O(|\mathcal{A}'|N)$, where $|\mathcal{A}'| \le M(M-1)$. Its benefit depends on the redundancy in the original fairness system and the scale of $|\mathcal{A}'|$.

A.3. Computing Fixed-Cardinality Optimal Revenues and Assortments in Lemma 3

Algorithm A.3 constructs a complete table of the fixed-cardinality optimal revenues $R_i^*(C)$ and one corresponding optimal assortment $\mathcal{S}_i^*(C)$, $C = 0, 1, \ldots, N, i \in \mathcal{M}$. The algorithm uses the same notation $I_i^{(z)}$, $\sigma_i^{(z)}$, and $\mathcal{B}_i^{(z)}(C)$ as Algorithm A.1 and extends them to multiple customer types.

Algorithm A.3 Polynomial Algorithm for Computing $R_i^*(C)$ and Fixed-Cardinality Optimal Assortments

Require: r_j, v_{ij}, and $\mathcal{S}_i^D$, for $i \in \mathcal{M}$, $j \in \mathcal{N}$.

Step 1: For each $i \in \mathcal{M}$, compute the intervals $[I_i^{(z)}, I_i^{(z+1)})$ and successively reorder the products to obtain $\sigma_i^{(z)}$, $z = 0, \ldots, Z_i$. Define $\Phi_i(z, C) = \sum_{j \in \mathcal{B}_i^{(z)}(C)} h_{ij}\left(I_i^{(z)}\right) - I_i^{(z)}$. For each $i \in \mathcal{M}$, set $\mathcal{S}_i^*(0) = \emptyset, R_i^*(0) = 0$. Let $D_i = |\mathcal{S}_i^D|, R_i^D = R_i(\mathcal{S}_i^D), z_i^D = \max\{z : I_i^{(z)} \le R_i^D\}$. Also record $\mathcal{S}_i^*(D_i) = \mathcal{S}_i^D$ and $R_i^*(D_i) = R_i^D$.

Step 2: For each $i \in \mathcal{M}$, set $z = 0$. For $C = 1, \ldots, D_i - 1$, repeat:

Step 2.1: While $z < Z_i$ and $\Phi_i(z+1, C) \ge 0$, update $z = z + 1$.

Step 2.2: Record $\mathcal{S}_i^*(C) = \mathcal{B}_i^{(z)}(C), R_i^*(C) = \frac{\sum_{j \in \mathcal{S}_i^*(C)} r_j v_{ij}}{1 + \sum_{j \in \mathcal{S}_i^*(C)} v_{ij}}$.

Step 3: For each $i \in \mathcal{M}$, set $z = z_i^D$. For $C = D_i + 1, \ldots, N$, repeat:

Step 3.1: While $z > 0$ and $\Phi_i(z, C) < 0$, update $z = z - 1$.

Step 3.2: Record $\mathcal{S}_i^*(C) = \mathcal{B}_i^{(z)}(C), R_i^*(C) = \frac{\sum_{j \in \mathcal{S}_i^*(C)} r_j v_{ij}}{1 + \sum_{j \in \mathcal{S}_i^*(C)} v_{ij}}$.

Ensure: The maximal revenue $R_i^*(C)$ and the optimal assortment $\mathcal{S}_i^*(C)$, for all $i \in \mathcal{M}$ and $C = 0, 1, \ldots, N$.

A.4. Alternative Randomized Fairness Constraints by Limiting Unfairness Probability

By limiting the probability that a customer from group i_1 experiences unfair treatment, we propose the following probability-based fairness constraint:

$$\sum_{i_2 \in \mathcal{M}} P_G(|\mathcal{S}_{i_2}| - |\mathcal{S}_{i_1}| > \alpha_{i_2,i_1}) \le \kappa_{i_1}, \forall i_1 \in \mathcal{M}. \tag{A.3}$$

We then study the fairness problem under the setting of two customer types. Compared to the problem under Constraints (8), this version is easier to solve, as it only involves the probability of unfairness rather than its expected severity. Proposition A.1 obtains the optimal solution under this constraint.

PROPOSITION A.1. *Under the fairness constraint* (A.3)*, if* $|\mathcal{S}_2^D| - |\mathcal{S}_1^D| \le \alpha_{2,1}$ *or* $\kappa_1 \ge 1$*, then an optimal randomized strategy* G^{F_p} *coincides with the deterministic fully customized strategy* $P_{G^{F_p}}(\mathcal{S}_1^D, \mathcal{S}_2^D) = 1$*. Otherwise, there exists an optimal randomized strategy* G^{F_p} *randomizing over at most two assortment pairs and satisfying:*

$$P_{G^{F_p}}(\mathcal{S}_1, \mathcal{S}_2) = \begin{cases} \kappa_1 & \text{if } (\mathcal{S}_1, \mathcal{S}_2) = (\mathcal{S}_1^D, \mathcal{S}_2^D), \\ 1 - \kappa_1 & \text{if } (\mathcal{S}_1, \mathcal{S}_2) = (\mathcal{S}_1^F, \mathcal{S}_2^F), \\ 0 & \text{otherwise}, \end{cases}$$

where $(\mathcal{S}_1^F, \mathcal{S}_2^F)$ *is the optimal solution of Problem* (DFA-2) *with tolerance levels* $\alpha_{1,2}$ *and* $\alpha_{2,1}$*.*

Proposition A.1 shows that if the fully customized assortments already satisfy the probability-based fairness requirement, then a purely deterministic strategy remains optimal. Otherwise, it mixes the fully customized assortment $(\mathcal{S}_1^D, \mathcal{S}_2^D)$ and the optimal assortment $(\mathcal{S}_1^F, \mathcal{S}_2^F)$ of Problem (DFA-2), where the latter can be obtained using Algorithm A.1. As a result, the optimal randomized strategy can be computed in polynomial time.

A.5. Polynomial Algorithm for Problem (RFA-2) in the Two-type Case

Algorithm A.4 provides a polynomial-time algorithm for solving Problem (RFA-2), where the notations $I_i^{(z)}$, $\sigma_i^{(z)}$, and related terms are defined as in Appendix A.1.

Algorithm A.4 Polynomial Algorithm for Assortment Fairness Problem under Constraints (8)

Require: Group size λ_i, fairness levels $\alpha_{1,2}$, $\alpha_{2,1}$, κ_1 and attraction values v_{ij} for $i=1,2$, $j\in\mathcal{N}$; r_j and $\mathcal{S}_i^D$.

Step 1: If $|\mathcal{S}_2^D|-|\mathcal{S}_1^D|-\alpha_{2,1}\le\kappa_1$, return the fully customized pair with probability one and stop. Otherwise, if $\kappa_1=0$, apply Algorithm A.1, return its pair with probability one, and stop. For $i=1,2$, compute the intervals $[I_i^{(z)},I_i^{(z+1)})$ and successively reorder the products to obtain $\sigma_i^{(z)}$, $z=0,\dots,Z_i$. Initiate $C_i=|\mathcal{S}_i^D|$, $i=1,2$. Let $\bar{z}_i=\max\{z\in\{0,\dots,Z_i\}|I_i^{(z)}\le\sum_{j\in\mathcal{S}_i^D} r_j\frac{v_{ij}}{1+\sum_{l\in\mathcal{S}_i^D}v_{il}}\}$, $i=1,2$.

Step 2: Repeat the following process while $C_1\le|\mathcal{S}_2^D|-\alpha_{2,1}$:

Step 2.1: Gradually decrease z_i from $\bar{z}_i$ until $\sum_{j\in\mathcal{B}_i^{(z_i)}(C_i)}h_{ij}(I_i^{(z_i)})\ge I_i^{(z_i)}$ holds. Then update $\bar{z}_i=z_i$, $i=1,2$.

Step 2.2: Record $\mathcal{S}_i(C_i)=\mathcal{B}_i^{(\bar{z}_i)}(C_i)$ and $R_i(\mathcal{S}_i(C_i))$, $i=1,2$.

Step 2.3: Update $C_1=C_1+1$, $C_2=C_2-1$.

Step 3: For every $\Delta\in\{\alpha_{2,1},\dots,|\mathcal{S}_2^D|-|\mathcal{S}_1^D|\}$, solve

$$R(\Delta)=\max_{C\in\mathbb{Z}}\left(\lambda_1R_1(\mathcal{S}_1(C))+\lambda_2R_2(\mathcal{S}_2(C+\Delta))\right),\text{ s.t. }|\mathcal{S}_1^D|\le C\le|\mathcal{S}_2^D|-\Delta$$

where the optimal solution is denoted by $C^*(\Delta)$.

Step 4: Solve the following problem:

$$\begin{aligned}\Pi=\max_{\Delta_1,\Delta_2}\quad&\frac{\Delta_2-\alpha_{2,1}-\kappa_1}{\Delta_2-\Delta_1}R(\Delta_1)+\frac{\alpha_{2,1}+\kappa_1-\Delta_1}{\Delta_2-\Delta_1}R(\Delta_2)\\ \text{s.t.}\quad&\Delta_1,\Delta_2\in\{\alpha_{2,1},\dots,|\mathcal{S}_2^D|-|\mathcal{S}_1^D|\},\quad\Delta_1<\Delta_2,\qquad\Delta_1\le\alpha_{2,1}+\kappa_1\le\Delta_2,\end{aligned}$$

where the optimal solution is denoted by Δ_1^*,Δ_2^*. Then the two assortment pairs of the optimal strategy G^F are $(\mathcal{S}_1^j=\mathcal{S}_1(C^*(\Delta_j^*)),\mathcal{S}_2^j=\mathcal{S}_2(C^*(\Delta_j^*)+\Delta_j^*))$, $j=1,2$.

Ensure: The optimal strategy G^F.

A.6. An Illustrative Example for Algorithm 2

We continue the example in Section 4.2.4 to illustrate the minimum s-t cut algorithm for $\Phi(\boldsymbol{\mu})$. We use the same input data and the fixed-cardinality revenues and assortments in Table 1. Suppose that, at a particular ellipsoid iteration, $\boldsymbol{\mu}=(0.1,0.04,0)$.

Step 1: We construct the graph shown in Figure A.1, with source s, sink t, and nodes $v_{i,k}$, where $i=1,2,3$ and $k=2,3,4$. Following Step 1.1, the marginal weights $w_{i,k}=\lambda_i(R_i^*(k)-R_i^*(k-1))$ are computed from Table 1 and coincide with those in Figure 1. They give $W^+\approx0.6111$ and $B=W^++1\approx1.6111$. Each box in Figure A.1 shows the node index, with its marginal weight below. Solid arcs incident to s or t are added in Step 1.2, with costs equal to the absolute values of the corresponding node weights. The remaining solid arcs are the consecutive-level arcs added in Step 1.3, each with cost B. Dashed arcs represent the fairness arcs added in Step 1.4. Tolerance levels equal to $N=4$ generate no arcs. Each arc is labeled with its cost.

Step 2: Using a minimum-cut algorithm (Ford et al. 2011), we obtain the source-side set $A_{\boldsymbol{\mu}}=\{s,v_{2,2},v_{3,2},v_{3,3},v_{3,4}\}$, shown by the shaded nodes in Figure A.1. Only two arcs leave this set: the sink arc $v_{2,2}\to t$ and the fairness arc $v_{3,4}\to v_{2,3}$, shown by the bold arcs. Their costs give

$$c_{\boldsymbol{\mu}}(A_{\boldsymbol{\mu}})\approx0.0196+0.0400=0.0596.$$

Step 3: We recover the cardinalities from the source-side nodes:

$$C_i^{\mu}=1+\sum_{k=2}^{4}\mathbf{1}\{v_{i,k}\in\mathcal{A}_{\boldsymbol{\mu}}\},\qquad(C_1^{\mu},C_2^{\mu},C_3^{\mu})=(1,2,4).$$

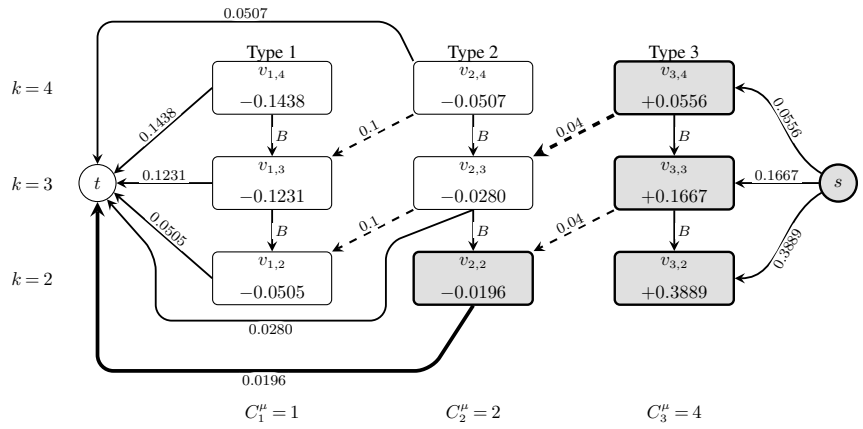


Figure A.1 Minimum-cut network for the illustrative example.

The corresponding objective value is $\Phi(\boldsymbol{\mu}) = \sum_{i=1}^{3} \lambda_i R_i^*(1) + W^+ - c_{\boldsymbol{\mu}}(A_{\boldsymbol{\mu}}) \approx 7.4404$.

A.7. Ellipsoid-based Algorithm in Proposition 2

Algorithm A.5 denotes the standard oracle-based exact ellipsoid procedure for rational linear programs; the rational rounding, exact termination, and solution recovery use the strong optimization procedure for well-described rational polyhedra in Grötschel et al. (1993). Steps 6-8 summarize its core iterations. At each iteration, it either identifies a constraint that excludes the current center or records an improved dual solution. The cardinality vectors associated with violated constraints are retained for the subsequent recovery of a randomized assortment policy.

Specifically, Steps 1-3 prepare the fixed-cardinality information and express the dual problem in a common vector form. Step 3 collects the dual variables into $\boldsymbol{x} = (\eta, \mu_1, \ldots, \mu_M)^\top$. The vector $\boldsymbol{q}$ contains the objective coefficients, while $\boldsymbol{a}(\boldsymbol{C})$ and $b(\boldsymbol{C})$ contain the coefficients and right-hand side of the constraint indexed by $\boldsymbol{C}$. Thus, the dual problem is written as

$$\min_{\boldsymbol{x} \geq 0} \left\{ \boldsymbol{q}^\top \boldsymbol{x} : \boldsymbol{a}(\boldsymbol{C})^\top \boldsymbol{x} \geq b(\boldsymbol{C}),\ \forall \boldsymbol{C} \in \mathcal{D}^M \right\}.$$

Steps 4 and 5 initialize a feasible benchmark and the ellipsoid search region. The fully customized revenue $\overline{F}$ bounds $F(\boldsymbol{C})$ from above for every $\boldsymbol{C}$, making $\boldsymbol{x}^{\text{best}} = (\overline{F}, 0, \ldots, 0)^\top$ dual feasible. Throughout the algorithm, $\boldsymbol{x}^{\text{best}}$ denotes the best dual-feasible solution found so far, and $\text{OBJ} = \boldsymbol{q}^\top \boldsymbol{x}^{\text{best}}$ denotes its objective value. The set $\mathcal{C}_v$ stores the cardinality vectors used for primal recovery and is initialized with $\boldsymbol{C}^0 = (D_1, \ldots, D_1)$. Because $U_i(\boldsymbol{C}^0) = 0$ for every i, this initialization ensures that the subsequent restricted primal problem is feasible. For integer tolerance levels, the bound $B_{\text{dual}} = \max\{1, \overline{F}\}$ guarantees that $[0, B_{\text{dual}}]^d$ contains at least one optimal dual solution, as shown in the proof of Proposition 2. Step 5 constructs an initial ellipsoid containing this box, where $\boldsymbol{x}^0$ is its center and Q^0 determines its size and shape.

Step 6.1 screens the current center before invoking the minimum-cut oracle. If $\boldsymbol{q}^\top \boldsymbol{x}^t \geq \text{OBJ}$, the center cannot improve the current benchmark, so the algorithm chooses the objective-cut direction $\boldsymbol{g}^t = -\boldsymbol{q}$. Otherwise, it checks the nonnegativity of η^t and the multipliers μ_i^t. A negative coordinate yields the direction $\boldsymbol{g}^t = \boldsymbol{e}_j$, where $\boldsymbol{e}_j$ is the unit vector associated with that coordinate. Steps 6.2 and 6.3 use the minimum-cut oracle to check all cardinality constraints simultaneously and update the feasible benchmark. For the current multipliers $\boldsymbol{\mu}^t$, Algorithm 2 returns a vector $\boldsymbol{C}^t$ such that $\Phi(\boldsymbol{\mu}^t) - \eta^t = b(\boldsymbol{C}^t) - \boldsymbol{a}(\boldsymbol{C}^t)^\top \boldsymbol{x}^t$. The algorithm records $\boldsymbol{C}^t$ in $\mathcal{C}_v$ at every such call. If this quantity is positive, the constraint indexed by $\boldsymbol{C}^t$ is violated and the algorithm sets $\boldsymbol{g}^t = \boldsymbol{a}(\boldsymbol{C}^t)$. Otherwise, all cardinality constraints are satisfied, so the center is dual feasible. Because this center has already passed the objective check, Step 6.3 updates $\boldsymbol{x}^{\text{best}}$ and OBJ and then uses an objective cut to continue searching for an improvement.

Algorithm A.5 Ellipsoid-based Algorithm in Proposition 2

Require: r_j, v_{ij}, λ_i, $\alpha_{m,i}$, κ_i.

Step 1: Apply Algorithm A.3 to compute $R_i^*(C)$ and assortment $\mathcal{S}_i^*(C)$, for every $i \in \mathcal{M}$ and $C = 0, 1, \ldots, N$. Set $R_i^*(0) = 0,\ \mathcal{S}_i^*(0) = \varnothing$.

Step 2: Define $F(\boldsymbol{C}) = \sum_{i\in\mathcal{M}} \lambda_i R_i^*(C_i),\ U_i(\boldsymbol{C}) = \sum_{m\in\mathcal{M}} (C_m - C_i - \alpha_{m,i})^+$.

Step 3: Let $d = M + 1$, and define $x = (\eta, \mu_1, \ldots, \mu_M)^\top$, $q = (1, \kappa_1, \ldots, \kappa_M)^\top$, $a(\boldsymbol{C}) = (1, U_1(\boldsymbol{C}), \ldots, U_M(\boldsymbol{C}))^\top,\ b(\boldsymbol{C}) = F(\boldsymbol{C})$.

Step 4: Define the upper bound $\overline{F} = \sum_{i\in\mathcal{M}} \lambda_i R_i^*(|\mathcal{S}_i^D|)$. Then $x^{\text{best}} := (\overline{F}, 0, \ldots, 0)^\top$ is dual feasible. Define $\text{OBJ} = \overline{F}$. Let $\boldsymbol{C}^0 = (|\mathcal{S}_1^D|, \ldots, |\mathcal{S}_1^D|)$ and initialize $\mathcal{C}_v := \{\boldsymbol{C}^0\}$.

Step 5: Compute $B_{\text{dual}} = \max\{1, \overline{F}\}$ such that at least one optimal dual solution belongs to $[0, B_{\text{dual}}]^d$. Initialize $x^0 := \frac{B_{\text{dual}}}{2}\mathbf{1}_d,\ Q^0 := \frac{dB_{\text{dual}}^2}{4} I_d,\ t := 0$. Thus the initial ellipsoid $\mathcal{E}^0 = \{y \in \mathbb{R}^d : (y - x^0)^\top (Q^0)^{-1}(y - x^0) \le 1\}$ contains $[0, B_{\text{dual}}]^d$.

Step 6: Repeat Steps 6.1-8 until the stopping criterion of the exact ellipsoid procedure is met. Write $x^t = (\eta^t, \mu_1^t, \ldots, \mu_M^t)^\top$.

Step 6.1: If $q^\top x^t \ge \text{OBJ}$, the current center cannot improve the objective. Set the objective-cut direction $g^t \leftarrow -q$. Go to Step 7.

Else if $\eta^t < 0$, let $e_0 = (1, 0, \ldots, 0)^\top \in \mathbb{R}^d$, and set $g^t \leftarrow e_0$. Go to Step 7.

Else if $\mu_i^t < 0$ for at least one $i \in \mathcal{M}$, select $i^t \leftarrow \min\{i \in \mathcal{M} : \mu_i^t < 0\}$. Let e_{i^t} be the unit vector corresponding to μ_{i^t}, and set $g^t \leftarrow e_{i^t}$. Go to Step 7.

Step 6.2: Apply Algorithm 2 to obtain a maximizer $\boldsymbol{C}^t$ of $\Phi(\boldsymbol{\mu}^t)$ and the exact value $\Phi(\boldsymbol{\mu}^t)$. Update $\mathcal{C}_v \leftarrow \mathcal{C}_v \cup \{\boldsymbol{C}^t\}$. If $\Phi(\boldsymbol{\mu}^t) > \eta^t$, the constraint associated with C^t is violated: $a(C^t)^\top x^t = \eta^t + \sum_{i\in\mathcal{M}} \mu_i^t U_i(C^t) < F(C^t) = b(C^t)$. Set $g^t \leftarrow a(C^t)$. Go to Step 7. Otherwise, go to Step 6.3.

Step 6.3: The current center x^t is dual feasible. Update $x^{\text{best}} \leftarrow x^t,\ \text{OBJ} \leftarrow q^\top x^t$. To search for a strictly better dual solution, set $g^t \leftarrow -q$.

Step 7: Update the ellipsoid center by $x^{t+1} \leftarrow x^t + \frac{1}{d+1} \frac{Q^t g^t}{\sqrt{(g^t)^\top Q^t g^t}}$. Update the ellipsoid matrix by $Q^{t+1} \leftarrow \frac{d^2}{d^2-1}\left(Q^t - \frac{2}{d+1} \frac{Q^t g^t (g^t)^\top Q^t}{(g^t)^\top Q^t g^t}\right)$.

Step 8: Update $t \leftarrow t + 1$.

Ensure: $\mathcal{C}_v$, OBJ and the final solution x^{best}.

Steps 7 and 8 update the search region and go into the next iteration. According to the ellipsoid method, in every branch, the chosen direction $\boldsymbol{g}^t$ defines the retained halfspace

$$\mathcal{H}^t = \left\{\boldsymbol{y} : (\boldsymbol{g}^t)^\top (\boldsymbol{y} - \boldsymbol{x}^t) \ge 0\right\},$$

which contains every dual-feasible point that improves the current best objective value. Step 7 updates the center $\boldsymbol{x}^t$ and the positive-definite shape matrix $\boldsymbol{Q}^t$ so that the next ellipsoid contains the intersection of the current ellipsoid and $\mathcal{H}^t$, while reducing its volume. Step 8 increments the iteration counter. The final set $\mathcal{C}_v$ is then used to construct the restricted primal problem.

Appendix B. Proofs

We first construct the following lemma to facilitate the proofs.

LEMMA B.2. *Consider a single customer type. Let* $R(S)=\frac{\sum_{j\in S} r_j v_j}{1+\sum_{l\in S} v_l}$. *For any* $S\subseteq\mathcal{N}$ *and* $j_0\notin S$, *we have*

$$\begin{aligned} R(S\cup\{j_0\}) &= \tfrac{1+\sum_{l\in S} v_l}{1+\sum_{l\in S\cup\{j_0\}} v_l}\tfrac{\sum_{j\in S} r_j v_j}{1+\sum_{l\in S} v_l}+\tfrac{v_{j_0}}{1+\sum_{l\in S\cup\{j_0\}} v_l} r_{j_0} \\ &= \tfrac{1+\sum_{l\in S} v_l}{1+\sum_{l\in S\cup\{j_0\}} v_l} R(S)+\tfrac{v_{j_0}}{1+\sum_{l\in S\cup\{j_0\}} v_l} r_{j_0}, \end{aligned}$$

where $R(S\cup\{j_0\})$ *is a linear combination of* $R(S)$ *and* r_{j_0}.

LEMMA B.3. *For any* $C\in\mathcal{N}$ *and* $i\in\mathcal{M}$ *there always exists a threshold* $z'\in\{0,\dots,Z_i\}$ *such that* $\sum_{j\in\mathcal{B}_i^{(z)}(C)} h_{ij}(I_i^{(z)})\geq I_i^{(z)}$ *for any* $0\leq z\leq z'$ *and* $\sum_{j\in\mathcal{B}_i^{(z)}(C)} h_{ij}(I_i^{(z)})< I_i^{(z)}$ *for any* $z'<z\leq Z_i$. *Moreover, the threshold* z' *records the position of* $R_i^*(C)$, *that is* $R_i^*(C)\in[I_i^{(z')}, I_i^{(z'+1)})$. *Here,* $I_i^{(Z_i+1)}=+\infty$, *and* $z'=0$ *when* $Z_i=0$.

Proof of Lemma B.3: Recall that $\max_{S:|S|=C}\sum_{j\in S} h_{ij}(I_i^{(z)}) = \sum_{j\in\mathcal{B}_i^{(z)}(C)} h_{ij}(I_i^{(z)})$ and $h_{ij}(R)$ decreases in R. Therefore, $\max_{S:|S|=C}\sum_{j\in S} h_{ij}(R)$ also decreases in R and $\max_{S:|S|=C}\sum_{j\in S} h_{ij}(I_i^{(z)}) - I_i^{(z)}$ decreases in z. Since $\max_{S:|S|=C}\sum_{j\in S} h_{ij}(I_i^{(0)}) - I_i^{(0)} = \max_{S:|S|=C}\sum_{j\in S} r_j v_{ij}\geq 0$, z' always exists as the largest passing index in $\{0,\dots,Z_i\}$.

It indicates that $R_i^*(C)\geq I_i^{(z')}$ otherwise

$$R_i^*(C)=\max_{S:|S|=C}\sum_{j\in S} h_{ij}(R_i^*(C))>\max_{S:|S|=C}\sum_{j\in S} h_{ij}(I_i^{(z')})\geq I_i^{(z')}>R_i^*(C)$$

and leads to a contradiction. The equality follows from Equation (B.4), and the inequalities follow from the definition of z'.

Then we will prove $R_i^*(C)\leq I_i^{(z'+1)})$. If $z'=Z_i$, then $R_i^*(C)<I_i^{(Z_i+1)}=+\infty$ because all product revenues are finite. Otherwise, assume $z'<Z_i$. If $R_i^*(C)\geq I_i^{(z'+1)}$, then

$$R_i^*(C)\geq I_i^{(z'+1)}>\max_{S:|S|=C}\sum_{j\in S} h_{ij}(I_i^{(z'+1)})\geq\max_{S:|S|=C}\sum_{j\in S} h_{ij}(R_i^*(C))=R_i^*(C),$$

which is a contradiction. Then we prove $R_i^*(C)\in[I_i^{(z')}, I_i^{(z'+1)})$. □

B.1. Proofs of Section 4

Proof of Lemma 2: We have

$$\begin{aligned} R_i^*(C) &= \max_{S:|S|=C}\frac{\sum_{j\in S} r_j v_{ij}}{1+\sum_{j\in S} v_{ij}} \\ &= \max_R\{R|R=\frac{\sum_{j\in S} r_j v_{ij}}{1+\sum_{j\in S} v_{ij}},\ \exists S,|S|=C\} \\ &= \max_R\{R|R=\sum_{j\in S}(r_j-R)v_{ij},\ \exists S,|S|=C\} \\ &= \max_R\{R|R=\max_{S:|S|=C}\sum_{j\in S}(r_j-R)v_{ij}\}, \end{aligned} \tag{B.4}$$

where $\mathcal{S}_i^*(C)$ represents the optimal assortment. Define the linear function $h_{ij}(R)$ to be $h_{ij}(R)=(r_j-R)v_{ij}$. Thus $\mathcal{S}_i^*(C)$ is the optimal solution of $\max_{S:|S|=C}\sum_{j\in S} h_{ij}(R_i^*(C))$, which equals to the summation of the top C largest $h_{ij}(R_i^*(C))$, $j\in\mathcal{N}$. Then we prove the unimodal results by analyzing $C\geq|\mathcal{S}_i^D|$ and $C\leq|\mathcal{S}_i^D|$, respectively.

The case $C \geq |\mathcal{S}_i^D|$: Now we prove that $R_i^*(C)$ decreases when $C \geq |\mathcal{S}_i^D|$. First, we show that when $C > |\mathcal{S}_i^D|$ and for all $j \in \mathcal{N} \backslash \mathcal{S}_i^*(C)$, $h_{ij}(R_i^*(C)) \leq 0$. Otherwise, if $j_0 \notin \mathcal{S}_i^*(C)$ and $h_{j_0}(R_i^*(C)) > 0$, it indicates that all the $h_{ij}(R_i^*(C)) \geq h_{j_0}(R_i^*(C)) > 0$ for $j \in \mathcal{S}_i^*(C)$ based on the construction rule of $\mathcal{S}_i^*(C)$. Then

$$\begin{aligned} R_i^*(|\mathcal{S}_i^D|) &\geq R_i^*(C) = \max_{S:|S|=C} \textstyle\sum_{j\in S} h_{ij}(R_i^*(C)) \\ &= \max_{S:|S|=|\mathcal{S}_i^D|} \textstyle\sum_{j\in S} h_{ij}(R_i^*(C)) + \sum_{j\in\mathcal{S}_i^*(C)\backslash\mathcal{S}_i^D} h_{ij}(R_i^*(C)) \\ &> \max_{S:|S|=|\mathcal{S}_i^D|} \textstyle\sum_{j\in S} h_{ij}(R_i^*(C)) \\ &\geq \max_{S:|S|=|\mathcal{S}_i^D|} \textstyle\sum_{j\in S} h_{ij}(R_i^*(|\mathcal{S}_i^D|)) = R_i^*(|\mathcal{S}_i^D|), \end{aligned}$$

where the first inequality is due to the definition of $\mathcal{S}_i^D$; the second inequality is because the second term $\sum_{j\in\mathcal{S}_i^*(C)\backslash\mathcal{S}_i^D} h_{ij}(R_i^*(C))$ is positive; the last inequality is because $h_{ij}(R)$ decreases in R and $R_i^*(C) \leq R_i^*(|\mathcal{S}_i^D|)$. It leads to a contradiction.

Second, we show that when $C = |\mathcal{S}_i^D|$ and for all $j \in \mathcal{N} \backslash \mathcal{S}_i^*(C)$, $h_{ij}(R_i^*(C)) \leq 0$. When $C = |\mathcal{S}_i^D|$, the problem is equivalent to the problem without any cardinality constraints, then

$$R_i^*(|\mathcal{S}_i^D|) = \max_S R(S) = \max_R\{R|R = \max_S \sum_{j\in S} h_{ij}(R)\} = \max_R\{R|R = \sum_{j:h_{ij}(R)\geq 0} h_{ij}(R)\}, \quad \text{(B.5)}$$

where the second equality is obtained similar to the reformulation process in Equation (B.4). Therefore, we also have $h_{ij}(R_i^*(|\mathcal{S}_i^D|)) \leq 0$ for all $j \in \mathcal{N} \backslash \mathcal{S}^*(|\mathcal{S}_i^D|)$.

Therefore, when $C \geq |\mathcal{S}_i^D|$, we have $\max_{k:k\notin\mathcal{S}_i^*(C)} h_k(R_i^*(C)) \leq 0$. It leads to

$$R_i^*(C) = \sum_{j\in\mathcal{S}_i^*(C)} h_{ij}(R_i^*(C)) \geq \max_{k:k\notin\mathcal{S}_i^*(C)} h_k(R_i^*(C)) + \sum_{j\in\mathcal{S}_i^*(C)} h_{ij}(R_i^*(C)) = \max_{S:|S|=C+1} \sum_{j\in S} h_{ij}(R_i^*(C)).$$

Note that $\max_{S:|S|=C+1} \sum_{j\in S} h_{ij}(R))$ decreases in R, then $R_i^*(C) \geq R_i^*(C+1)$ otherwise

$$R_i^*(C) < R_i^*(C+1) = \max_{S:|S|=C+1} \sum_{j\in S} h_{ij}(R_i^*(C+1)) \leq \max_{S:|S|=C+1} \sum_{j\in S} h_{ij}(R_i^*(C)),$$

which leads to a contradiction. Then we show that $R_i^*(C) \geq R_i^*(C+1)$ when $C \geq |\mathcal{S}_i^D|$ and finish the proof.

The case $C \leq |\mathcal{S}_i^D|$: Now we prove that $R_i^*(C)$ increases when $C \leq |\mathcal{S}_i^D|$. First, we show that when $C < |\mathcal{S}_i^D|$ and for all $j \in \mathcal{S}_i^*(C)$, $h_{ij}(R_i^*(C)) \geq 0$. Otherwise, if $j_0 \in \mathcal{S}_i^*(C)$ and $h_{j_0}(R_i^*(C)) < 0$, it indicates that all the $h_{ij}(R_i^*(C)) < 0$ for $j \notin \mathcal{S}_i^*(C)$ based on the construction rule of $\mathcal{S}_i^*(C)$. Then

$$\begin{aligned} R_i^*(|\mathcal{S}_i^D|) &\geq R_i^*(C) = \max_{S:|S|=C} h_{ij}(R_i^*(C)) \\ &> \max_{S:|S|=|\mathcal{S}_i^D|} \textstyle\sum_{j\in S} h_{ij}(R_i^*(C)) \\ &\geq \max_{S:|S|=|\mathcal{S}_i^D|} \textstyle\sum_{j\in S} h_{ij}(R_i^*(|\mathcal{S}_i^D|)) = R_i^*(|\mathcal{S}_i^D|), \end{aligned}$$

where the second inequality is due to $h_{ij}(R_i^*(C)) < 0$ for all $j \notin \mathcal{S}_i^*(C)$ and the last inequality is because $h_{ij}(R)$ decreases in R. It leads to a contradiction.

Second, we show that when $C = |\mathcal{S}_i^D|$ and for all $j \in \mathcal{S}_i^*(C)$, $h_{ij}(R_i^*(C)) \geq 0$. It is because $R_i^*(|\mathcal{S}_i^D|) = \max_R\{R|R = \sum_{j:h_{ij}(R)\geq 0} h_{ij}(R)\}$ based on Equation (B.5).

Therefore, when $C \leq |\mathcal{S}_i^D|$, we have $\min_{k:k\in\mathcal{S}_i^*(C)} h_k(R_i^*(C)) \geq 0$ and thus

$$R_i^*(C) = \sum_{j\in\mathcal{S}_i^*(C)} h_{ij}(R_i^*(C)) \geq \sum_{j\in\mathcal{S}_i^*(C)} h_{ij}(R_i^*(C)) - \min_{k:k\in\mathcal{S}_i^*(C)} h_k(R_i^*(C)) = \max_{S:|S|=C-1} \sum_{j\in S} h_{ij}(R_i^*(C)).$$

Note that $\max_{S:|S|=C-1} \sum_{j\in S} h_{ij}(R))$ decreases in R, then $R_i^*(C) \geq R_i^*(C-1)$ otherwise

$$R_i^*(C) < R_i^*(C-1) = \max_{S:|S|=C-1} \sum_{j\in S} h_{ij}(R_i^*(C-1)) \leq \max_{S:|S|=C-1} \sum_{j\in S} h_{ij}(R_i^*(C)),$$

which leads to a contradiction. Then we show that $R_i^*(C-1) \leq R_i^*(C)$ when $C \leq |\mathcal{S}_i^D|$ and finish the proof. □

Proof of Theorem 1: When $|\mathcal{S}_2^D| - |\mathcal{S}_1^D| \leq \alpha_{2,1}$, $(\mathcal{S}_1^D, \mathcal{S}_2^D)$ satisfies the fairness constraints and becomes the optimal solution automatically. Then we only need to consider the case with $|\mathcal{S}_2^D| - |\mathcal{S}_1^D| > \alpha_{2,1}$. Define $R_i^*(C)$ to be the optimal revenue under the fixed cardinality C and $\mathcal{S}_i^*(C)$ to be the corresponding optimal assortment for type-i customers.

We first show that an optimal pair can be chosen such that $|\mathcal{S}_1^D| \leq |\mathcal{S}_1^F| \leq |\mathcal{S}_2^D|$ and $|\mathcal{S}_1^D| \leq |\mathcal{S}_2^F| \leq |\mathcal{S}_2^D|$. For any optimal pair, set $\widehat{C}_i = \min\{|\mathcal{S}_2^D|, \max\{|\mathcal{S}_1^D|, |\mathcal{S}_i^F|\}\}$ and replace $\mathcal{S}_i^F$ by $\mathcal{S}_i^*(\widehat{C}_i)$. Since $\widehat{C}_i - \widehat{C}_j \leq \max\{|\mathcal{S}_i^F| - |\mathcal{S}_j^F|, 0\} \leq \alpha_{i,j}$ and each cardinality moves toward $|\mathcal{S}_i^D|$, feasibility is preserved and revenue does not decrease by Lemma 2. In particular, the following adjustments have this property:

(a) $|\mathcal{S}_1^F|, |\mathcal{S}_2^F| < |\mathcal{S}_1^D|$: the solution $(\mathcal{S}_1^D, \mathcal{S}_2^*(|\mathcal{S}_1^D|))$ is feasible and does not decrease revenue.
(b) $|\mathcal{S}_1^F| < |\mathcal{S}_1^D| \leq |\mathcal{S}_2^F|$: the solution $(\mathcal{S}_1^D, \mathcal{S}_2^F)$ is feasible and does not decrease revenue.
(c) $|\mathcal{S}_1^F| \in [|\mathcal{S}_1^D|, |\mathcal{S}_2^D|]$ and $|\mathcal{S}_2^F| > |\mathcal{S}_2^D|$: the solution $(\mathcal{S}_1^F, \mathcal{S}_2^D)$ is feasible and does not decrease revenue.
(d) $|\mathcal{S}_1^F|, |\mathcal{S}_2^F| > |\mathcal{S}_2^D|$: the solution $(\mathcal{S}_1^*(|\mathcal{S}_2^D|), \mathcal{S}_2^D)$ is feasible and does not decrease revenue.

Then we may choose $|\mathcal{S}_1^F| \leq |\mathcal{S}_2^F|$, since otherwise $(\mathcal{S}_1^*(|\mathcal{S}_2^F|), \mathcal{S}_2^F)$ is also feasible and does not decrease revenue since $R_1^*(C)$ is nonincreasing in C and $R_2^*(C)$ is nondecreasing in C for $C \in [|\mathcal{S}_1^D|, |\mathcal{S}_2^D|]$ according to Lemma 2.

Moreover, suppose that $|\mathcal{S}_2^F| - |\mathcal{S}_1^F| < \alpha_{2,1}$. If $|\mathcal{S}_1^F| > |\mathcal{S}_1^D|$, we replace $\mathcal{S}_1^F$ by $\mathcal{S}_1^*(|\mathcal{S}_1^F| - 1)$. Otherwise, $|\mathcal{S}_1^F| = |\mathcal{S}_1^D|$ and $|\mathcal{S}_2^F| < |\mathcal{S}_1^D| + \alpha_{2,1} < |\mathcal{S}_2^D|$. In this case, we replace $\mathcal{S}_2^F$ by $\mathcal{S}_2^*(|\mathcal{S}_2^F| + 1)$. Each replacement preserves feasibility and does not decrease revenue by Lemma 2. Repeating these adjustments gives $|\mathcal{S}_2^F| - |\mathcal{S}_1^F| = \alpha_{2,1}$ while preserving optimality, which completes the proof. □

Proof of Corollary 1: Based on Theorem 1, if $|\mathcal{S}_2^D| - |\mathcal{S}_1^D| \leq \alpha_{2,1}$, Corollary 1 always holds. Then we consider the case $|\mathcal{S}_2^D| - |\mathcal{S}_1^D| > \alpha_{2,1}$, where we have $|\mathcal{S}_1^F| - |\mathcal{S}_2^F| = -\alpha_{2,1}$ and $|\mathcal{S}_2^F| \in [|\mathcal{S}_1^D| + \alpha_{2,1}, |\mathcal{S}_2^D|]$. Note that $\lambda_2 = 1 - \lambda_1$. By the selection rule stated before the corollary, we choose the optimal pair characterized in Theorem 1 with the smallest cardinality for type 1 and type 2.

We first show $|\mathcal{S}_1^F|$ decreases in λ_1. Otherwise, assume there exists λ_1, λ_1' such that $\lambda_1 > \lambda_1'$ and $|\mathcal{S}_1^F(\lambda_1)| > |\mathcal{S}_1^F(\lambda_1')|$, where $\mathcal{S}_i^F(\lambda_1)$ is the optimal solution for group i under the parameter λ_1. Then it indicates $R_1^*(|\mathcal{S}_1^F(\lambda_1)|) \leq R_1^*(|\mathcal{S}_1^F(\lambda_1')|)$ and $R_2^*(|\mathcal{S}_1^F(\lambda_1)| + \alpha_{2,1}) \geq R_2^*(|\mathcal{S}_1^F(\lambda_1')| + \alpha_{2,1})$ based on the unimodal property in Lemma 2 and the cardinality range in Theorem 1, where $R_i^*(C)$ is the optimal revenue for the group i when the assortment cardinality is C. Based on Theorem 1, the assortment problem is equivalent to

$$\max_{C\in[|\mathcal{S}_1^D|, |\mathcal{S}_2^D|-\alpha_{2,1}]} \lambda_1 R_1^*(C) + \lambda_2 R_2^*(C + \alpha_{2,1}).$$

Since $\mathcal{S}_i^F(\lambda_1)$ is the optimal assortment for λ_1, $i = 1, 2$. Then we have

$$\lambda_1 R_1^*(|\mathcal{S}_1^F(\lambda_1)|) + (1-\lambda_1) R_2^*(|\mathcal{S}_1^F(\lambda_1)| + \alpha_{2,1}) > \lambda_1 R_1^*(|\mathcal{S}_1^F(\lambda_1')|) + (1-\lambda_1) R_2^*(|\mathcal{S}_1^F(\lambda_1')| + \alpha_{2,1}).$$

Note that the inequality holds strictly based on our solution-selection rule and $|\mathcal{S}_1^F(\lambda_1)| > |\mathcal{S}_1^F(\lambda_1')|$. The above inequality is equivalent to

$$\lambda_1 \left(R_1^*(|\mathcal{S}_1^F(\lambda_1')|) - R_1^*(|\mathcal{S}_1^F(\lambda_1)|)\right) < (1-\lambda_1) \left(R_2^*(|\mathcal{S}_1^F(\lambda_1)| + \alpha_{2,1}) - R_2^*(|\mathcal{S}_1^F(\lambda_1')| + \alpha_{2,1})\right). \tag{B.6}$$

Similarly, $\mathcal{S}_i^F(\lambda_1')$ is the optimal assortment for λ_1', $i=1,2$, indicating

$$\lambda_1' R_1^*(|\mathcal{S}_1^F(\lambda_1')|)+(1-\lambda_1')R_2^*(|\mathcal{S}_1^F(\lambda_1')|+\alpha_{2,1})\geq \lambda_1' R_1^*(|\mathcal{S}_1^F(\lambda_1)|)+(1-\lambda_1')R_2^*(|\mathcal{S}_1^F(\lambda_1)|+\alpha_{2,1}),$$

which is equivalent to

$$\lambda_1'\left(R_1^*(|\mathcal{S}_1^F(\lambda_1)|)-R_1^*(|\mathcal{S}_1^F(\lambda_1')|)\right)\leq(1-\lambda_1')\left(R_2^*(|\mathcal{S}_1^F(\lambda_1')|+\alpha_{2,1})-R_2^*(|\mathcal{S}_1^F(\lambda_1)|+\alpha_{2,1})\right). \tag{B.7}$$

Add the left-hand sides and right-hand sides of the inequalities (B.6) and (B.7) to obtain a new inequality:

$$(\lambda_1-\lambda_1')\left(R_1^*(|\mathcal{S}_1^F(\lambda_1')|)-R_1^*(|\mathcal{S}_1^F(\lambda_1)|)\right)<(\lambda_1'-\lambda_1)\left(R_2^*(|\mathcal{S}_1^F(\lambda_1)|+\alpha_{2,1})-R_2^*(|\mathcal{S}_1^F(\lambda_1')|+\alpha_{2,1})\right).$$

It leads to a contradiction because the left-hand side is nonnegative while the right-hand side is nonpositive. Therefore, $|\mathcal{S}_1^F|$ decreases in λ_1, which automatically leads to $|\mathcal{S}_1^F|$ and $|\mathcal{S}_2^F|$ decreases in λ_1 (increases in λ_2). □

Proof of Theorem 2: We first prove Algorithm A.1 can achieve the optimal solution. If $|\mathcal{S}_2^D|-|\mathcal{S}_1^D|\leq\alpha_{2,1}$, Step 1 returns the optimal fully customized pair by Theorem 1. Otherwise, based on Theorem 1 and Problem (A.1), the optimal assortment of Problem (DFA-2) can be derived by

$$\max_{C\in\mathbb{Z}_+\cap[|\mathcal{S}_1^D|,|\mathcal{S}_2^D|-\alpha_{2,1}]}\lambda_1 R_1^*(C)+\lambda_2 R_2^*(C+\alpha_{2,1}).$$

Obviously, Algorithm B.6 can return the optimal solution. Denote the position of $R_i^*(C)$ to be $z_i'(C)$ for ease of further discussion.

Next, we only need to prove that, for every enumerated C, we may choose optimal assortments satisfying $\mathcal{S}_1^*(C)=\mathcal{B}_1^{(\bar{z}_1)}(C)$, $\mathcal{S}_2^*(C+\alpha_{2,1})=\mathcal{B}_2^{(\underline{z}_2)}(C+\alpha_{2,1})$, where $\mathcal{B}_1^{(\bar{z}_1)}(C)$ and $\mathcal{B}_2^{(\underline{z}_2)}(C+\alpha_{2,1})$ are identified in Step 2.1 and used in Step 2.2 of Algorithm A.1. We prove the result by induction. Note that based on the unimodal property in Lemma 2, the position $z_i'(C)$, which can be derived by Lemma B.3, increases in C when $C\leq|\mathcal{S}_i^D|$ and decreases in C when $C\geq|\mathcal{S}_i^D|$. Therefore, $z_1'(C)$ decreases in C and $z_2'(C+\alpha_{2,1})$ increases in C for $C\in[|\mathcal{S}_1^D|,|\mathcal{S}_2^D|-\alpha_{2,1}]$.

Algorithm B.6 The brief algorithm for Problem (DFA-2)

Ensure: $\mathcal{S}_i^F$, $i=1,2$.

Step 1: Set $C=|\mathcal{S}_1^D|$ and $\Pi_{max}=-\infty$.

Step 2: Repeat the following process while $C\leq|\mathcal{S}_2^D|-\alpha_{2,1}$:

Step 2.1: Derive the optimal assortment $\mathcal{S}_1^*(C)$ and $\mathcal{S}_2^*(C+\alpha_{2,1})$.

Step 2.2: Set $\Pi_{temp}=\lambda_1 R_1(\mathcal{S}_1^*(C))+\lambda_2 R_2(\mathcal{S}_2^*(C+\alpha_{2,1}))$. If $\Pi_{temp}>\Pi_{max}$, record $\mathcal{S}_1^F=\mathcal{S}_1^*(C)$ and $\mathcal{S}_2^F=\mathcal{S}_2^*(C+\alpha_{2,1})$ and update $\Pi_{max}=\Pi_{temp}$.

Step 2.3: Update $C=C+1$.

We first establish the base case at $C=|\mathcal{S}_1^D|$. In Step 1, $\bar{z}_1=z_1'(|\mathcal{S}_1^D|)$ because $R_1(\mathcal{S}_1^D)=R_1^*(|\mathcal{S}_1^D|)$. In contrast, $\underline{z}_2=0$ is only a valid starting index, since $I_2^{(0)}=0\leq R_2^*(|\mathcal{S}_1^D|+\alpha_{2,1})$. The first execution of Step 2.1 leaves $\bar{z}_1$ unchanged and increases z_2 while $\sum_{j\in\mathcal{B}_i^{(z_2)}(C)}h_{ij}(I_i^{(z_2)})-I_i^{(z_2)}\geq 0$. By Lemma B.3, it stops exactly at $z_2'(|\mathcal{S}_1^D|+\alpha_{2,1})$, either because the inequality fails or because $z_2=Z_2$. Thus, both recorded indices are correct after Step 2.1 of the first iteration.

We next establish the induction step. Assume that, before Step 2.1 at a cardinality $C>|\mathcal{S}_1^D|$, we have $\bar{z}_1=z_1'(C-1)$ and $\underline{z}_2=z_2'(C+\alpha_{2,1}-1)$. By the monotonicity established

above, the former is an upper bound on $z_1'(C)$ and the latter is a lower bound on $z_2'(C+\alpha_{2,1})$. Decreasing z_1 until $\sum_{j\in\mathcal{B}_i^{(z_1)}(C)} h_{ij}(I_i^{(z_1)}) - I_i^{(z_1)} \geq 0$ therefore gives $\bar{z}_1 = z_1'(C)$. Increasing z_2 while $\sum_{j\in\mathcal{B}_i^{(z_2)}(C)} h_{ij}(I_i^{(z_2)}) - I_i^{(z_2)} \geq 0$ gives $\underline{z}_2 = z_2'(C+\alpha_{2,1})$. Both conclusions follow from Lemma B.3, and the guards keep every evaluated index within $\{0,\dots,Z_i\}$. Therefore, Step 2.1 correctly tracks both revenue positions at every iteration.

We finally recover the optimal assortments at these positions. By Equation (A.2), $\mathcal{B}_1^{(\bar{z}_1)}(C)$ and $\mathcal{B}_2^{(\underline{z}_2)}(C+\alpha_{2,1})$ attain $R_1^*(C)$ and $R_2^*(C+\alpha_{2,1})$, respectively. Hence, Step 2.2 evaluates the same revenue as Algorithm B.6. Since $\Pi_{max} = -\infty$, the first pair is recorded even if its revenue is zero. Enumerating all feasible C and retaining the best pair returns an exact optimal solution.

We analyze the complexity of Algorithm A.1. First, count the index movements in Step 2.1, including the first search for type 2. For each $C > |\mathcal{S}_1^D|$, z_1 decreases from $z_1'(C-1)$ to $z_1'(C)$. Its total number of downward movements is

$$\sum_{C=|\mathcal{S}_1^D|+1}^{|\mathcal{S}_2^D|-\alpha_{2,1}} \big(z_1'(C-1) - z_1'(C)\big) = z_1'(|\mathcal{S}_1^D|) - z_1'(|\mathcal{S}_2^D| - \alpha_{2,1}) \leq Z_1.$$

For type 2, the first iteration starts at zero. Including this initial search, the total number of upward movements is

$$z_2'(|\mathcal{S}_1^D| + \alpha_{2,1}) + \sum_{C=|\mathcal{S}_1^D|+1}^{|\mathcal{S}_2^D|-\alpha_{2,1}} \big(z_2'(C+\alpha_{2,1}) - z_2'(C+\alpha_{2,1}-1)\big) = z_2'(|\mathcal{S}_2^D|) \leq Z_2.$$

Thus, the number of operations in Step 2.1 is $O(Z_1+Z_2) = O(N^2)$.

For each type i, we record the current ordering $\sigma_i^{(z)}$ and the index l of each product $j_l^{i,z}$ in this ordering. We also record the values of $\sum_{j\in\mathcal{B}_i^{(z)}(C)} r_j v_{ij}$ and $\sum_{j\in\mathcal{B}_i^{(z)}(C)} v_{ij}$. When C increases by one, we add the contributions of $j_{C+1}^{i,z}$ to these two sums. When two neighboring products exchange their order, $\mathcal{B}_i^{(z)}(C)$ either remains unchanged or replaces one product by the other. If product j is removed and product k is added, we subtract $r_j v_{ij}$ and v_{ij} from the respective sums and add $r_k v_{ik}$ and v_{ik}. If the assortment does not change, both sums remain unchanged. Thus, each such update takes $O(1)$ time. After updating these sums, each test in Step 2.1 takes $O(1)$ time because $\sum_{j\in\mathcal{B}_i^{(z)}(C)} h_{ij}(R) = \sum_{j\in\mathcal{B}_i^{(z)}(C)} r_j v_{ij} - R\sum_{j\in\mathcal{B}_i^{(z)}(C)} v_{ij}$. The revenue in Step 2.2 is also computed from the same two sums in $O(1)$ time. For $z = 1,\dots,Z_i$, the orderings in the two intervals on either side of $I_i^{(z)}$ satisfy $\sum_{j\in\mathcal{B}_i^{(z-1)}(C)} h_{ij}(I_i^{(z)}) = \sum_{j\in\mathcal{B}_i^{(z)}(C)} h_{ij}(I_i^{(z)})$, because each pair of functions whose order changes at $I_i^{(z)}$ has equal values there. If several pairs intersect at the same $I_i^{(z)}$, we process all corresponding changes in the ordering. Since z_1 only decreases and z_2 only increases, each pair of nonparallel functions is processed at most once for each type. Thus, there are $O(N^2)$ exchanges in total. Recording at most $O(N)$ assortments, each containing at most N products, also takes $O(N^2)$ time. Therefore, after sorting the intersection records, the cardinality scan takes $O(N^2)$ time.

Finally, Step 1 computes and sorts the pairwise intersection records using the construction in Rusmevichientong et al. (2010, Section 2.1). There are $O(N^2)$ such records, so this requires $O(N^2 \log N)$ time per type. Constructing the initial ordering and the successive crossing updates does not exceed this bound. This preprocessing is performed once, independently of the cardinality scan. Including it gives the stated overall bound of $O(N^2 \log N)$. □

Proof of Lemma 3. Fix a customer type $i \in \mathcal{M}$. For any cardinality C, recall the reformulation $R_i^*(C) = \max_R \left\{ R : R \leq \max_{S\subseteq\mathcal{N}:|S|=C} \sum_{j\in S} h_{ij}(R) \right\}$, where $h_{ij}(R) = v_{ij}(r_j - R)$. For any fixed R, the inner maximization is obtained by choosing the C largest values of $h_{ij}(R)$, $j \in \mathcal{N}$.

By Lemma B.3, for each C there exists a threshold index, denoted by $z_i(C)$, such that $\Phi_i(z,C) \geq 0$ for all $z \leq z_i(C)$, and $\Phi_i(z,C) < 0$ for all $z > z_i(C)$. Moreover, $R_i^*(C) \in \left[I_i^{(z_i(C))}, I_i^{(z_i(C)+1)}\right)$. Based on previous definition, the order of the linear functions $h_{ij}(R)$, $j \in \mathcal{N}$, is fixed and is represented by $\sigma_i^{(z_i(C))}$ and for every $R \in [I_i^{(z_i(C))}, I_i^{(z_i(C)+1)})$, $\max_{S\subseteq\mathcal{N}:|S|=C} \sum_{j\in S} h_{ij}(R) = \sum_{j\in B_i^{(z_i(C))}(C)} h_{ij}(R)$. By the definition of h_{ij}, the equality above is equivalent to $R_i^*(C) = \frac{\sum_{j\in B_i^{(z_i(C))}(C)} r_j v_{ij}}{1+\sum_{j\in B_i^{(z_i(C))}(C)} v_{ij}}$. Thus, it remains only to show that Algorithm A.3 identifies $z_i(C)$ for every C. Based on the proof of Theorem 2, the position $z_i(C)$ is nondecreasing in C for $C \leq D_i$ and nonincreasing for $C \geq D_i$. Therefore, in steps 2 and 3, starting from the previously recorded position, the algorithm correctly records the positions for every C. In Step 2, the first search starts from zero, which is a valid lower bound; in Step 3, the first search starts from z_i^D, which is a valid upper bound. The guarded tests also cover $Z_i = 0$ and the last interval. The values at $C = 0$ and $C = D_i$ are recorded directly in Step 1.

It remains to analyze the running time. For each fixed i, the number of distinct positive intersection coordinates between nonparallel functions $h_{ij}(R)$, $j \in \mathcal{N}$, is at most $O(N^2)$ because two different lines have at most one intersection. Thus, the interval construction and product reordering in Step 1 take $O(N^2 \log N)$ time per type, including the sorting of the intersection records. In Step 2, the index z only increases, so the total number of upward movements over all $C < D_i$ is at most Z_i. In Step 3, the index z only decreases, so the total number of downward movements over all $C > D_i$ is also at most Z_i. Thus, for each customer type, the total number of interval-position updates is $O(Z_i) = O(N^2)$. Therefore, the total running time, including interval construction and product reordering for all M types, is $O\big(M\big(N^2 \log N\big)\big) = O(MN^2 \log N)$. This completes the proof. □

Proof of Lemma 4. Among all optimal solutions, choose $\boldsymbol{C}^F$ to minimize $L(\boldsymbol{C}) := \sum_{i\in\mathcal{M}} |C_i - D_i|$. Suppose that some $C_i^F \notin [|\mathcal{S}_1^D|, |\mathcal{S}_M^D|]$, and define $\widehat{C}_i := \min\{|\mathcal{S}_M^D|, \max\{|\mathcal{S}_1^D|, C_i^F\}\}$. Since truncation to an interval is nondecreasing and 1-Lipschitz, for every $i_1, i_2 \in \mathcal{M}$, $i_1 \neq i_2$,

$$\widehat{C}_{i_1} - \widehat{C}_{i_2} \leq \max\{C_{i_1}^F - C_{i_2}^F, 0\} \leq \alpha_{i_1,i_2}.$$

Hence, $\widehat{\boldsymbol{C}}$ is feasible. Moreover, truncation moves each C_i^F weakly toward $|\mathcal{S}_i^D|$, so Lemma 2 implies $R_i^*(\widehat{C}_i) \geq R_i^*(C_i^F)$ for every i. Thus, $\widehat{\boldsymbol{C}}$ is also optimal, but $L(\widehat{\boldsymbol{C}}) < L(\boldsymbol{C}^F)$, contradicting the choice of $\boldsymbol{C}^F$. Therefore, $|\mathcal{S}_1^D| \leq C_i^F \leq |\mathcal{S}_M^D|$ for every $i \in \mathcal{M}$. □

Proof of Lemma 5. We first restrict the cardinality range without changing the optimal value. By Lemma 4, adding the bounds $D_1 \leq C_i \leq D_M$ for all $i \in \mathcal{M}$ to Problem (3) does not change its optimal value. It therefore suffices to establish a correspondence between this restricted problem and Problem (4)-(7).

First, we show that every feasible binary solution yields a feasible cardinality vector with the same objective value. Given any feasible solution $\boldsymbol{y}$ to Problem (4)-(7), define $C_i = D_1 + \sum_{k=D_1+1}^{D_M} y_{i,k},\ i \in \mathcal{M}$. By Constraint (5), the entries $y_{i,k}$ for each type consist of consecutive ones followed by zeros. Hence, $C_i \in \{D_1, \ldots, D_M\}$ and $y_{i,k} = \mathbf{1}_{C_i \geq k}$ for every defined level k. Suppose that $C_{i_1} - C_{i_2} > \alpha_{i_1,i_2}$ for some $i_1, i_2 \in \mathcal{M}$, $i_1 \neq i_2$ for a contradiction. Define $k = C_{i_1}$, then $y_{i_1,k} = 1$ and $y_{i_2,k-\alpha_{i_1,i_2}} = 0$ because $k - \alpha_{i_1,i_2} > C_{i_2}$. This contradicts Constraint (6). Therefore, $\boldsymbol{C}$ is feasible for the restricted problem and hence for Problem (3). The construction also preserves the objective value. By the definition of $w_{i,k}$, for every $i \in \mathcal{M}$,

$$\lambda_i R_i^*(D_1) + \sum_{k=D_1+1}^{D_M} w_{i,k} y_{i,k} = \lambda_i R_i^*(D_1) + \lambda_i \sum_{k=D_1+1}^{C_i} \big(R_i^*(k) - R_i^*(k-1)\big) = \lambda_i R_i^*(C_i).$$

Summing over all customer types shows that $\boldsymbol{C}$ attains the same objective value in Problem (3) as $\boldsymbol{y}$ does in Problem (4)-(7).

Conversely, every feasible cardinality vector in the restricted problem yields a feasible binary solution with the same objective value. Given any feasible solution $\boldsymbol{C}$ to Problem (3) satisfying $D_1 \le C_i \le D_M$ for all $i \in \mathcal{M}$, define

$$y_{i,k} = 1\{C_i \ge k\}, \qquad i \in \mathcal{M}, \quad k = D_1 + 1, \dots, D_M.$$

These variables satisfy Constraints (5) and (7). For every index appearing in Constraint (6), if $y_{i_1,k} = 0$, the inequality holds automatically. Otherwise, $C_{i_1} \ge k$, and the feasibility of $\boldsymbol{C}$ implies

$$C_{i_2} \ge C_{i_1} - \alpha_{i_1,i_2} \ge k - \alpha_{i_1,i_2}.$$

Consequently, $y_{i_2,k-\alpha_{i_1,i_2}} = 1$, so Constraint (6) also holds. Thus, $\boldsymbol{y}$ is feasible for Problem (4)-(7). The same telescoping identity above shows that $\boldsymbol{y}$ attains the same objective value as $\boldsymbol{C}$.

The two constructions establish that the restricted problem and Problem (4)-(7) have the same optimal value. Since the restriction does not change the optimal value of Problem (3), the claimed equivalence follows. Moreover, the first construction recovers an optimal solution to Problem (3) from any optimal solution to Problem (4)-(7). □

Proof of Theorem 3. We first reduce the correctness proof to the maximum-weight closure step. Recall from Section 4.2.2 that the original assortment problem is exactly equivalent to Problem (4)-(7), where $R_i^*(C)$ and $\mathcal{S}_i^*(C)$ have already been computed for all relevant cardinalities in Step 1.

We first show that every feasible binary solution $\boldsymbol{y}$ to Constraints (5)-(7) induces a closed node set in G. Define

$$\mathcal{U}(\boldsymbol{y}) = \{(i,k) \in V : y_{i,k} = 1\}$$

and we will show $U(\boldsymbol{y})$ is closed. Consider any arc $(i,k) \to (i,k-1)$ in G. If $(i,k) \in \mathcal{U}(\boldsymbol{y})$, then $y_{i,k} = 1$, and Constraint (5) implies $y_{i,k-1} = 1$; hence, $(i,k-1) \in \mathcal{U}(\boldsymbol{y})$. Similarly, consider any arc $(i_1,k) \to (i_2, k-\alpha_{i_1,i_2})$ in G. If $(i_1,k) \in \mathcal{U}(\boldsymbol{y})$, then Constraint (6) implies $y_{i_2,k-\alpha_{i_1,i_2}} = 1$, so $(i_2, k-\alpha_{i_1,i_2}) \in \mathcal{U}(\boldsymbol{y})$. Therefore, no arc leaves $U(\boldsymbol{y})$, and $U(\boldsymbol{y})$ is closed.

We next show that every closed node set $U \subseteq V$ induces a feasible binary solution to Constraints (5)-(7). Define

$$y_{i,k}(U) = \begin{cases} 1, & (i,k) \in \mathcal{U}, \\ 0, & (i,k) \notin \mathcal{U}, \end{cases} \qquad i \in \mathcal{M}, \quad k = |\mathcal{S}_1^D| + 1, \dots, |\mathcal{S}_M^D|.$$

For every $k = |\mathcal{S}_1^D| + 2, \dots, |\mathcal{S}_M^D|$, the graph contains the arc $(i,k) \to (i,k-1)$. Since $\mathcal{U}$ is closed, $y_{i,k}(\mathcal{U}) = 1$ implies $y_{i,k-1}(\mathcal{U}) = 1$, and hence Constraint (5) holds. At the lowest level $k = |\mathcal{S}_1^D| + 1$, no additional prefix constraint is required. Equivalently, the inequality at this level is automatically satisfied because $C_i \ge |\mathcal{S}_1^D|$. For every index appearing in Constraint (6), the graph contains the arc $(i_1,k) \to (i_2, k-\alpha_{i_1,i_2})$. Closedness therefore implies $y_{i_1,k}(U) \le y_{i_2,k-\alpha_{i_1,i_2}}(U)$. Thus, $\boldsymbol{y}(U)$ satisfies Constraints (5)-(7). The two constructions are inverse to each other, so feasible binary solutions to Constraints (5)-(7) and closed node sets in G are in one-to-one correspondence.

We then show that the corresponding objective values differ only by the constant term in (4). For any feasible $\boldsymbol{y}$ and its corresponding closed set $\mathcal{U}(\boldsymbol{y})$, the total node weight is $\sum_{(i,k)\in\mathcal{U}(\boldsymbol{y})} w_{i,k} = \sum_{i\in\mathcal{M}} \sum_{k=|\mathcal{S}_1^D|+1}^{|\mathcal{S}_M^D|} w_{i,k} y_{i,k}$. The objective in (4) is equal to this weight plus $\sum_{i\in\mathcal{M}} \lambda_i R_i^*(|\mathcal{S}_1^D|)$, which is independent of $\boldsymbol{y}$. Therefore, a binary vector is optimal for Problem (4)-(7) if and only if its corresponding closed node set has maximum total weight. Hence, Step 3 of Algorithm 1 solves Problem (4)-(7) exactly.

We can now establish the correctness of Algorithm 1 for the original assortment problem. Recall from Section 4.2.2 that Problem (4)-(7) is an exact reformulation of Problem (3). Consequently, the maximum-weight closed set U^* obtained in Step 3 determines an optimal binary vector $\boldsymbol{y}^*$ and the associated optimal cardinality vector C^F. Since Step 1 has computed $\mathcal{S}_i^*(C)$ for every

relevant i and C, Step 3 returns $\mathcal{S}_i^F = \mathcal{S}_i^*(C_i^F)$ for every $i \in \mathcal{M}$, which is an optimal solution to Problem (DFA-M).

Finally, we establish the running time. Computing all fixed-cardinality revenues and assortments requires $O(MN^2 \log N)$ time. The closure graph contains $M\big(|\mathcal{S}_M^D| - |\mathcal{S}_1^D|\big) = O(MN)$ nodes. The arcs $(i,k) \to (i,k-1)$ contribute $O(MN)$ arcs, while the fairness constraints contribute $O(M^2N)$ arcs. Hence, the graph contains $O(M^2N)$ arcs. The standard reduction from maximum-weight closure to minimum s-t cut adds only two nodes and $O(MN)$ arcs (Picard 1976). Using Dinitz's maximum-flow algorithm, which runs in $O(|V|^2|E|)$ time (Dinitz 2006), the closure step therefore requires $O((MN)^2M^2N) = O(M^4N^3)$ time. Graph construction and solution recovery are linear in the graph size and are dominated by this term. The total running time is thus $O(MN^2 \log N + M^4N^3) = O(M^4N^3)$. $\square$

B.2. Remaining Proofs

Due to the page limit, online Appendix C includes the remaining proofs, which includes proofs for Sections 5 and 6, as well as the proofs for Appendix A. It is available at

`https://github.com/hyh50033/Appendix_C_Assortment_with_Customer_Fairness.git`

References

Dinitz Y (2006) Dinitz'algorithm: The original version and even's version. Theoretical Computer Science: Essays in Memory of Shimon Even, 218–240 (Springer).

Floyd RW (1962) Algorithm 97: shortest path. Communications of the ACM 5(6):345–345.

Ford LR, Orlin JB, Bland RG, Fulkerson D (2011) Flows in Networks (Princeton University Press).

Grötschel M, Lovász L, Schrijver A (1993) Geometric Algorithms and Combinatorial Optimization (Berlin, Heidelberg: Springer).

Picard JC (1976) Maximal closure of a graph and applications to combinatorial problems. Management science 22(11):1268–1272.

Rusmevichientong P, Shen ZJM, Shmoys DB (2010) Dynamic assortment optimization with a multinomial logit choice model and capacity constraint. Operations Research 58(6):1666–1680.